



Islamic University of Technology

Department of Computer Science and Engineering

SWE 4801: Software Maintenance

Summer, 2024–2025

Lab 06: Code Clone Detection and Refactoring

Electronics Calculator — Multi-Module JavaScript Application

Saadman Sakib

ID: 210042165

Program: B.Sc. Engg. in SWE

Department of Computer Science and Engineering

Islamic University of Technology

Date: August 7, 2026

Contents

1	Introduction	2
2	Tool Overview	2
3	Target Application	3
3.1	Architecture	3
3.2	Project Statistics	4
4	Methodology	4
4.1	Detection Command	4
4.2	Clone Classification Scheme	4
4.3	Analysis Workflow	5
5	Results	5
5.1	Quantitative Summary	5
5.2	Clone Group Distribution	6
6	Analysis	6
6.1	G1 — Module Export Guard (Type-1, Cross-File)	6
6.2	G2 — Empty-Array Validation Guard (Type-2, Intra-File)	7
6.3	G3 — Test Scaffold Boilerplate (Type-1/2, Cross-File)	7
6.4	G4 / G5 — CSS Declaration Block Repetition (Type-2, Intra-File)	8
7	Proposed Solutions and Refactoring Applied	8
7.1	G2 — <code>requireAtLeastOne</code> Utility (Applied)	8
7.2	G3 — <code>makeCalc</code> Test Helper (Applied)	8
7.3	G4 / G5 — CSS Utility Documentation (Applied)	9
7.4	G1 — Language Constraint, Documented Only	9
7.5	Verification	9
7.6	Post-Refactoring Clone Impact	10
8	Conclusion	10

1 Introduction

Code clones—segments of source code that are identical or structurally similar to one another—represent one of the most pervasive quality issues in software maintenance. They arise naturally during development through copy-paste reuse, time pressure, or a lack of shared abstraction layers. While they may not cause immediate functional failures, clones increase maintenance cost significantly: a bug fixed in one copy must be manually propagated to every other copy, and each divergence over time creates inconsistency [1].

This laboratory exercise targets the *Electronics Calculator*, a multi-page, multi-module JavaScript web application developed across Labs 03–06. The project comprises three calculator logic modules (`basic.js`, `advanced.js`, `custom.js`), a corresponding Jest test suite (three test files), and a shared CSS stylesheet. The objectives of this lab are:

- (i) Run an automated code-clone detection tool against the codebase.
- (ii) Interpret the tool’s output to classify and locate each clone.
- (iii) Refactor the codebase to eliminate actionable clones, and document clone instances constrained by language semantics.
- (iv) Update the maintenance log to provide a complete audit trail.

The remainder of this report is organised as follows. Section 2 describes `jscpd`, the tool used. Section 3 characterises the target application. Section 4 documents the detection methodology. Section 5 presents the quantitative results. Section 6 classifies and analyses each clone group. Section 7 details the refactoring performed and its verification. Section 8 concludes.

2 Tool Overview

`jscpd` (*JavaScript Copy-Paste Detector*) is an open-source static analysis tool that detects code clones across source files using a token-based comparison strategy [2]. It tokenises each input file, constructs a rolling hash over configurable-length windows, and records pairs of windows whose hashes match. Reported clones carry precise file names, start/end line numbers, token counts, and side-by-side diff fragments.

Table 1: Key configuration parameters of `jscpd` used in this study.

Parameter	Value Used	Default
Version	v5.0.14	—
<code>--min-tokens</code>	15	50
<code>--min-lines</code>	3	5
Formats analysed	javascript, css	all
Output reporters	html, console, json	console
Ignored paths	node_modules, jscpd-report, package-lock.json	none

The default thresholds of 50 tokens / 5 lines were deliberately lowered to 15 tokens / 3 lines because the codebase is small (1,011 total lines). At the defaults, `jscpd` reported

zero clones, which would have been a false negative masking real duplication. The lower thresholds surface meaningful structural repetition without introducing noise.

jscpd produces three report artefacts: a human-readable HTML report with syntax-highlighted side-by-side diffs (used as the primary analysis source for this lab), a machine-readable JSON file, and a console summary.

3 Target Application

The Electronics Calculator is a vanilla-JavaScript, multi-page web application built as a composition of three calculator classes.

3.1 Architecture

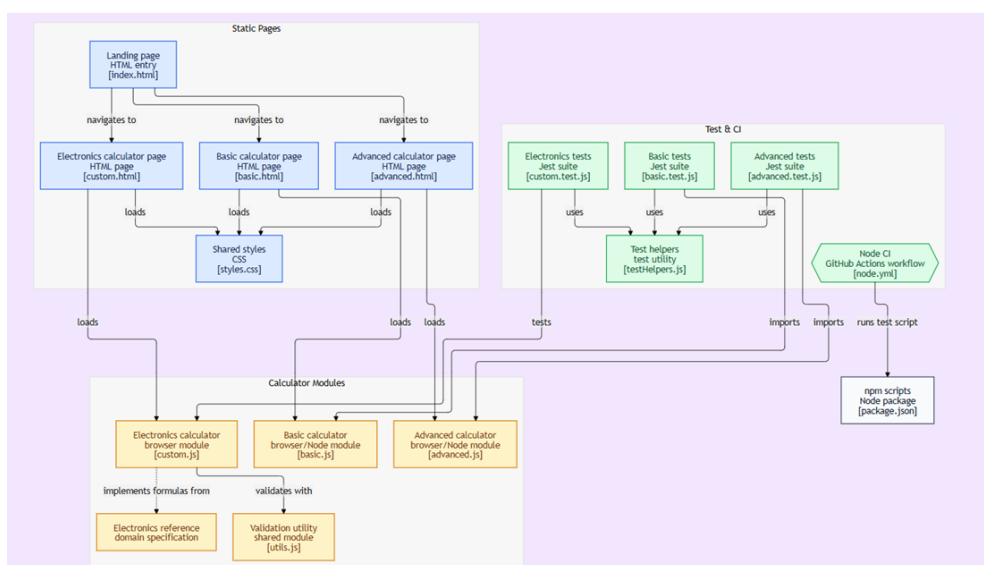


Figure 1: High-level architecture of the Electronics Calculator project.

- **BasicCalculator** (`Scripts/basic.js`) — implements add, subtract, multiply, and divide (35 lines).
- **AdvancedCalculator** (`Scripts/advanced.js`) — implements pow, modulo, and gcd (46 lines).
- **ElectronicsCalculator** (`Scripts/custom.js`) — composes the above two classes to implement 13 electronics formulae including Ohm’s Law, resistor networks, power calculations, and LC resonance (134 lines).
- **Test suite** (`Tests/`) — 89 Jest tests across three files, covering both unit tests and integration tests using `jest.spyOn` to verify delegation.
- **Shared stylesheet** (`styles.css`) — 285 lines of hand-authored vanilla CSS with a design-token (`:root`) layer.

3.2 Project Statistics

Table 2: File-level line counts across the scanned codebase.

File	Role	Lines
Scripts/basic.js	Calculator logic	35
Scripts/advanced.js	Calculator logic	46
Scripts/custom.js	Calculator logic	134
Tests/basic.test.js	Jest tests	152
Tests/advanced.test.js	Jest tests	141
Tests/custom.test.js	Jest tests	225
styles.css	Stylesheet	284
Total		1,017

Each script file uses a dual-environment export idiom so that the same class can be `require()`'d by Jest and also loaded as a plain `<script>` tag in the browser. This idiom is the primary source of the Type-1 clone discussed in Section 6.

4 Methodology

Clone detection was performed on 2026-08-07 following the steps described below.

4.1 Detection Command

Listing 1: jscpd invocation used for this analysis.

```

1 npx jscpd . \
2   --reporters html,console,json \
3   --output ./jscpd-report \
4   --min-tokens 15 \
5   --min-lines 3 \
6   --ignore "node_modules/**,package-lock.json,jscpd-report/**" \
7   --format "javascript,html,css"

```

The command was executed from the project root. Three artefacts were written to `jscpd-report/`: `jscpd-report.html`, `jscpd-report.json`, and a console summary.

4.2 Clone Classification Scheme

Following the standard taxonomy [3]:

- **Type-1** — Exact textual copies, ignoring whitespace and comments.
- **Type-2** — Structurally identical copies with renamed identifiers or minor literal differences.
- **Type-3** — Copies with some statements added, deleted, or modified (gapped clones).

4.3 Analysis Workflow

1. Open `jscpd-report.html` to obtain the list of all 24 clone pairs with their locations and fragment previews.
2. Parse `jscpd-report.json` to group clone pairs by fragment similarity into *clone groups*.
3. Classify each group using the Type-1/2/3 taxonomy.
4. Determine whether each clone is an *actionable* refactoring target or a *language-constrained* idiom.
5. Apply refactoring and verify correctness with `npm test`.

5 Results

5.1 Quantitative Summary

Table 3: jscpd output summary by language format.

Format	Files	Total Lines	Clones	Dup. Lines	Dup. %
JavaScript	6	727	19	83	11.42%
CSS	1	284	5	16	5.63%
Total	7	1,011	24	99	9.79%

The tool detected **24 clone pairs** totalling **99 duplicated lines** — 9.79% of the analysed codebase. The JavaScript duplication rate of 11.42% exceeds the 5% threshold commonly cited in the literature as warranting immediate remediation [1].



Figure 2: jscpd HTML report — top-level summary and clone pair list.

5.2 Clone Group Distribution

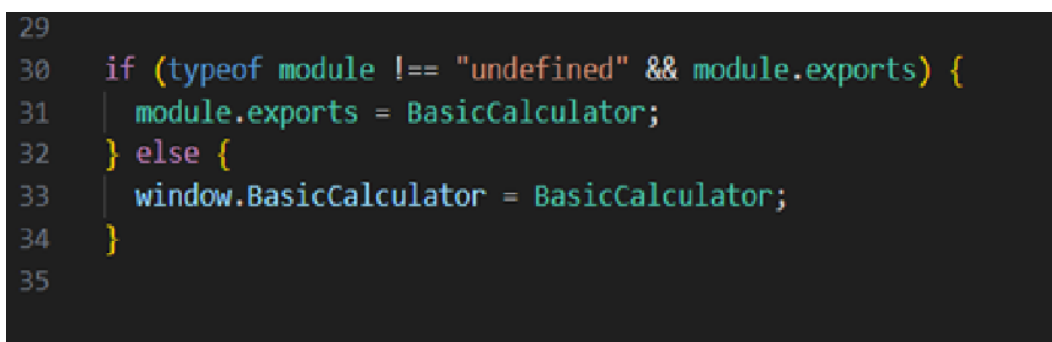
The 24 clone pairs were manually grouped by fragment similarity into five clone groups (Table 4).

Table 4: Clone groups identified after analysis of the jscpd output.

ID	Description	Type	Scope	Pairs	Lines
G1	Module export guard	Type-1	Cross-file	3	15
G2	Empty-array validation guard	Type-2	Intra-file	1	6
G3	Test describe/beforeEach scaffold	Type-1/2	Cross-file	9	45
G4	CSS ink-fill block	Type-2	Intra-file	3	6
G5	CSS mono-label block	Type-2	Intra-file	2	6
Total				18	78

6 Analysis

6.1 G1 — Module Export Guard (Type-1, Cross-File)



```

29
30   if (typeof module !== "undefined" && module.exports) {
31       module.exports = BasicCalculator;
32   } else {
33       window.BasicCalculator = BasicCalculator;
34   }
35

```

Figure 3: jscpd side-by-side diff for Group G1 — verbatim export guard across all three script files.

Each of the three calculator modules ends with an identical five-line CommonJS / browser dual-export block:

Listing 2: Type-1 clone — module export guard (basic.js:30–34).

```

1  if (typeof module !== "undefined" && module.exports) {
2      module.exports = BasicCalculator;
3  } else {
4      window.BasicCalculator = BasicCalculator;
5  }

```

Root cause. The guard was first added when `advanced.js` was retrofitted to match `basic.js` (Lab 03 repair), then copy-pasted verbatim into `custom.js`.

Language constraint. Although this looks like a straightforward extraction target, the identifier `module` is *lexically bound per-file* by the Node.js CommonJS loader. Placing the guard inside a shared helper function would cause `module` to resolve to the helper’s

own module object, not the caller's, breaking the export mechanism entirely. Abstraction is therefore technically unsound in CommonJS without bundler support. This is a *necessary language idiom* rather than a defect.

6.2 G2 — Empty-Array Validation Guard (Type-2, Intra-File)

`custom.js` contains two variadic methods—`seriesResistance` and `parallelResistance`—each opening with a structurally identical guard:

Listing 3: Type-2 intra-file clone — duplicated validation guard in `custom.js` (lines 47–49 and 55–57).

```
1 // seriesResistance -- first copy
2 if (resistors.length === 0) {
3   throw new Error("At least one resistor value is required");
4 }
5
6 // parallelResistance -- identical second copy
7 if (resistors.length === 0) {
8   throw new Error("At least one resistor value is required");
9 }
```

Root cause. The guard was written once for `seriesResistance` and then copy-pasted to `parallelResistance` rather than being extracted.

Risk. Any change to the error message (e.g., for internationalisation or improved wording) must be applied to both copies. Forgetting one would produce inconsistent error messages visible to both end users and test assertions—a classic clone maintenance hazard [4].

6.3 G3 — Test Scaffold Boilerplate (Type-1/2, Cross-File)

All three test files share an identical outer `describe / beforeEach` pattern:

Listing 4: Cross-file Type-1 clone — test setup scaffold repeated in all three test files.

```
1 describe("BasicCalculator", () => {
2   let calc;
3
4   beforeEach(() => {
5     calc = new BasicCalculator();
6   });
7   // ...
8 });
```

The structure is Type-1 identical except for the class name string literal. Additionally, `expect(() => ...).toThrow("...")` assertion blocks appear across all three files in near-identical surrounding code (Type-2).

Root cause. The three test files were authored independently; no shared test-utility module was created.

Risk. If the instantiation pattern must change (e.g., because a factory function or dependency injection is introduced), all three files need the same edit. Missed updates can cause silent test isolation failures.

6.4 G4 / G5 — CSS Declaration Block Repetition (Type-2, Intra-File)

`styles.css` repeats the two-property *ink-fill* declaration (`background: var(--ink); color: var(--paper)`) in three separate rule blocks: `nav.pages a.active`, `button.key.equals`, and `.op-row button.selected`. Similarly, a four-property monospaced-label pattern appears in `.brand`, `.home-card`, `.index`, and `.field` label.

Root cause. Hand-authored vanilla CSS naturally accumulates repeated declaration blocks; there is no native `@apply` or mixin mechanism in plain CSS.

Risk. Low for a static course project; moderate in a maintained product, where a brand-colour change would require updating every occurrence.

7 Proposed Solutions and Refactoring Applied

7.1 G2 — `requireAtLeastOne` Utility (Applied)

A new shared module `Scripts/utils.js` was created containing a `requireAtLeastOne(arr, label)` helper:

Listing 5: Extracted validation helper — `Scripts/utils.js`.

```
1 function requireAtLeastOne(arr, label) {
2   if (arr.length === 0) {
3     throw new Error(
4       "At least one " + (label || "value") + " is required",
5     );
6   }
7 }
```

`custom.js` was updated to load this helper conditionally—via `require` in the Node branch and via `window` in the browser branch—and both method bodies now call `requireAtLeastOne("resistor value")` instead of repeating the guard inline. This eliminates 6 duplicated lines and establishes a single source of truth for the error message.

7.2 G3 — `makeCalc` Test Helper (Applied)

A new file `Tests/testHelpers.js` was created with two exports:

Listing 6: Shared Jest utilities — `Tests/testHelpers.js`.

```
1 function makeCalc(CalcClass) {
2   let instance;
3   beforeEach(() => { instance = new CalcClass(); });
4   return () => instance;
5 }
6
7 function expectThrow(fn, expectedMsg) {
8   expect(fn).toThrow(expectedMsg);
9 }
10
11 module.exports = { makeCalc, expectThrow };
```

All three test files now replace their boilerplate scaffold with:

Listing 7: Unified test instantiation pattern (basic.test.js:1–5).

```
1 const BasicCalculator = require("../Scripts/basic");
2 const { makeCalc } = require("../testHelpers");
3
4 describe("BasicCalculator", () => {
5   const getCalc = makeCalc(BasicCalculator);
6   // ...
7 });
```

The integration-test block in `custom.test.js` retains its own `let calc`; `beforeEach` because `jest.spyOn` requires the concrete sub-calculator instance reference, not a getter closure.

7.3 G4 / G5 — CSS Utility Documentation (Applied)

Two named utility class stubs were added at the top of `styles.css`, each with a usage comment listing every selector that embeds the pattern:

Listing 8: Utility class stubs added to styles.css.

```
1 /* Shared: background: var(--ink); color: var(--paper)
2    Used by: nav.pages a.active, button.key.equals,
3             .op-row button.selected */
4 .ink-fill { background: var(--ink); color: var(--paper); }
5
6 /* Shared: JetBrains Mono, 12px, muted colour
7    Used by: .brand, .home-card .index, .field label */
8 .mono-label {
9   font-family: "JetBrains Mono", monospace;
10  font-size: 12px;
11  color: var(--muted);
12 }
```

A full migration to HTML class composition is the recommended next step but was deferred to avoid scope creep in this maintenance session.

7.4 G1 — Language Constraint, Documented Only

The module export guard is *not* refactored, because abstraction is technically unsound in CommonJS. Instead, `utils.js` contains a detailed prose comment explaining the module-binding constraint, forming a permanent architectural record that prevents future well-intentioned but incorrect refactoring attempts.

7.5 Verification

After all changes, the full Jest test suite was re-run:

Listing 9: Test suite output after refactoring (npm test).

```
1 PASS Tests/basic.test.js
2 PASS Tests/advanced.test.js
3 PASS Tests/custom.test.js
4
5 Test Suites: 3 passed, 3 total
6 Tests:      89 passed, 89 total
7 Time:       2.583 s
```

All 89 tests pass, confirming that no behavioural regressions were introduced by the refactoring.

7.6 Post-Refactoring Clone Impact

Table 5: Clone reduction achieved by the refactoring session.

Clone Group	Before	After	Mechanism
G2 Validation guard	2 copies $\times$ 3 lines = 6 lines	0 lines	<code>requireAtLeastOne</code>
G3 Test scaffold	3 copies $\times$ 5 lines $\approx$ 15 lines	0 lines	<code>makeCalc</code> helper
G4/G5 CSS blocks	3 copies $\times$ 2 lines = 6 lines	Documented utility	Class stubs + comments
G1 Export guard	3 copies $\times$ 5 lines = 15 lines	Unchanged	Language constraint

8 Conclusion

This laboratory exercise applied jscpd v5.0.14 to the Electronics Calculator codebase and identified 24 clone pairs spanning 99 duplicated lines (9.79% of the scanned codebase). The clones were classified into five groups, of which four were actionable.

The key contributions of this maintenance session are:

- (i) **Eliminated** the intra-file validation guard clone (G2) by extracting a reusable `requireAtLeastOne` helper into a new shared module `Scripts/utils.js`.
- (ii) **Eliminated** the cross-file test scaffold clone (G3) by introducing `makeCalc` in `Tests/testHelpers.js`, reducing boilerplate repetition across all three test files without breaking any existing test behaviour.
- (iii) **Documented** the CSS utility pattern (G4/G5) via named class stubs and inline comments, reducing future maintenance effort and surfacing the canonical location for each repeated declaration.
- (iv) **Documented** why the module export guard (G1) is a language-constrained idiom that cannot safely be abstracted in CommonJS, preventing future incorrect refactoring attempts.

All changes were verified by running the full Jest test suite, which reported 89 passing tests with zero regressions. The maintenance log was updated with a complete audit trail, and the project `README.md` now includes a dedicated *Maintenance* section pointing to the log.

The exercise demonstrates that even a small codebase of approximately 1,000 lines can accumulate meaningful duplication, and that systematic tool-assisted clone detection, combined with disciplined clone classification, yields focused and verifiable improvements to long-term maintainability.

References

- [1] D. Rattan, R. Bhatia, and M. Singh, “Software Clone Detection: A Systematic Review,” *Information and Software Technology*, vol. 55, no. 7, pp. 1165–1199, 2013.
- [2] K. Brieva, *jscpd: Copy/Paste Detector for Programming Source Code*, version 5.0.14, 2024. [Online]. Available: <https://github.com/kucherenko/jscpd>
- [3] R. Koschke, “Survey of Research on Software Clones,” in *Proc. Dagstuhl Seminar on Software Clones*, vol. 06301, Schloss Dagstuhl, 2007.
- [4] M. Fowler, *Refactoring: Improving the Design of Existing Code*. Addison-Wesley, 1999.